

AI Settings Configuration Guide

AI Interface — Frappe App

May 15, 2026

CONTENTS

AI Settings — Configuration Guide	
1. What AI Settings Controls	
2. The Five Fields	
2.1 Default Provider (required)	
2.2 Default Model (required)	
2.3 API Call Timeout (seconds)	
2.4 Max Output Tokens	
2.5 Max PDF Pages (Vision section)	
2.6 Enable Call Logging (default: ON)	
3. Recommended Defaults by Project Stage	
Pilot / proof-of-concept	
Production (low volume, <1000 calls/day)	
Production (high volume, 10000+ calls/day)	
4. Pitfalls	
5. Verifying Your Setup	
6. When to Change These Settings vs Per-Call Overrides	

AI Settings — Configuration Guide

Audience: System Managers configuring AI Interface for a Frappe site. **Time to set up:** ~5 minutes. **Last reviewed:** 2026-05-15

1. What AI Settings Controls

AI Settings is a Frappe *Single* doctype — there's exactly one record per site. It holds the **site-wide defaults** that every AI call falls back on when the consumer app doesn't specify its own values.

The dispatcher resolves each parameter in this order:

```
caller override    →    template override    →    AI Settings default
(highest)                                     (lowest)
```

That means: a developer can hard-code a specific model in their `generate()` call, a prompt template can pin its preferred model, and everything else uses what you set here. Setting a sane default here makes every consumer app “just work” without per-call configuration.

2. The Five Fields

2.1 DEFAULT PROVIDER (*REQUIRED*)

What it is: Which `AI Provider` doctype record runs the call.

How to pick:

Scenario	Choose
You have a Claude Pro/Max subscription	An Anthropic-type provider with Auth Token (paste OAuth token from <code>claude setup-token</code>)
You have an Anthropic API account with credits	An Anthropic-type provider with API Key (<code>sk-ant-api03-*</code>)
You want zero config — already logged into Claude Code CLI	A Claude Code -type provider
You're testing only	Any of the above — make a second provider with <code>enabled = 0</code> to A/B switch

Trade-offs:

- **OAuth (Pro plan):** free if you're already paying for Pro, fast (~1-2s latency), but shared with your interactive Claude Code usage.
- **API Key:** separate billing, no quota interaction with Pro, highest rate limits, best for production at scale.
- **Claude Code CLI:** zero credential management, but ~10x slower (subprocess overhead), reports inflated token counts (includes reasoning).

Recommendation: Start with OAuth + Pro for development. Switch to API Key once production traffic justifies it.

2.2 DEFAULT MODEL (*REQUIRED*)

What it is: The model ID string within the chosen provider. Free-text, but must match a `model_id` in the provider's `models` child table (populated by clicking “Fetch Models” on the provider doc).

Anthropic family (current as of 2026):

Model	When to use	Approx cost/M input	Approx cost/M output
<code>claude-opus-4-7</code>	Highest-stakes generation (legal drafting, complex reasoning, multi-turn analysis)	\$15	\$75
<code>claude-sonnet-4-6</code>	Default recommendation. Balanced quality + cost. Vision, summarization, drafting, NL → data queries	\$3	\$15
<code>claude-haiku-4-5</code>	High-volume + low-stakes (classification, simple extraction, intent detection)	\$0.80	\$4

How to choose:

- Default to **Sonnet**. Drop to Haiku only when you've measured Sonnet is overkill for a specific consumer workflow (use a template-level override, not a global change).
- Upgrade to Opus only when you've seen Sonnet fail at a specific task type. Cost is ~5x.
- Per-template overrides exist for a reason — use them. Example: keep `default_model = claude-sonnet-4-6`, but pin `email_draft` template to Haiku.

2.3 API CALL TIMEOUT (SECONDS)

What it is: How long a single provider HTTP call can run before the dispatcher raises.

Recommended values:

Use case	Setting
General text generation	120s (default)
Long-form drafting (reports, multi-page output)	240s
Vision on large PDFs (20+ pages)	300s
Synchronous UI-facing calls	30-60s (fail fast, user is waiting)

Behind the scenes: The timeout applies to the *provider HTTP call* inside `_execute_ai_call`. The total user-perceived wait can be longer in async mode (queue wait + execution). In sync mode this is roughly the worst-case wait.

Symptoms of wrong values:

- Too low: vision extractions on large PDFs fail intermittently. Check `AI Call Log` for `Failed` status with timeout traceback.
- Too high: a stuck provider holds a Frappe worker for the full duration. Lowers system throughput.

2.4 MAX OUTPUT TOKENS

What it is: Hard ceiling on the response length the provider may generate per call.

Cost impact: Output tokens are 3-5x the price of input tokens. This is the most direct lever on per-call cost.

Sizing guide:

Output type	Tokens	Approx words
One-liner (intent tag, classification)	100	~75
Short reply / summary	500	~375
Email draft	1500	~1100
Default (4096)	4096	~3000
Long-form report	8192-16000	6000-12000
Anthropic Sonnet 4.x max	64000	~48000
Anthropic Opus 4.x max	32000	~24000

Recommendation: Leave at 4096. Override per-call (`max_tokens=` parameter) when you need more.

Note: This is a *ceiling*, not a target. The model usually stops naturally at `stop_reason=end_turn` well before this limit. Setting 16000 doesn't cost you anything if the model only generates 500 tokens.

2.5 MAX PDF PAGES (VISION SECTION)

What it is: When `extract(file_url=...)` receives a PDF, `pdf2image` rasterizes the first N pages and sends each as an image to the vision model.

Cost per page: ~1500 input tokens (Anthropic vision pricing).

Sizing guide:

Document type	Pages needed
Sales invoice (single page)	1-3
Purchase order	2-5
Lab/test report	3-10
Default (10)	covers ~95% of business docs
Contract / agreement	30-50
Annual report	100+ (don't — chunk it manually)

Recommendation: Default 10 is fine. If your primary use case is contracts, raise to 30. Don't go above 50 — at that point manual chunking is cheaper and more accurate.

Caller can't override this globally — it's a site-wide cap by design, to prevent a runaway `extract()` call from sending a 500-page PDF and burning \$20 in tokens.

2.6 ENABLE CALL LOGGING (DEFAULT: ON)

What it is: Whether every call creates an `AI Call Log` record with full prompt, response, tokens, cost, latency, and status.

Keep it ON unless:

Reason to turn off	What you lose
Data retention policy forbids storing prompts	Async delivery breaks (consumers poll the log)
Disk space is critical	Cost auditing, debugging, latency monitoring
Privacy: prompts contain PII	Better: use field-level masking in the consumer app before calling

Auto-cleanup: Logs auto-clear after **90 days** via `default_log_clearing_doctypes` in `hooks.py`. Adjust there if you need a different retention.

Reporting from logs:

```
# Daily AI cost
SELECT DATE(creation), SUM(cost), COUNT(*)
FROM `tabAI Call Log`
WHERE status = 'Completed'
GROUP BY DATE(creation);
```

3. Recommended Defaults by Project Stage

PILOT / PROOF-OF-CONCEPT

- Provider: Anthropic + OAuth (Pro plan)
- Model: `claude-sonnet-4-6`
- Timeout: 120s
- Max output: 4096
- PDF pages: 10
- Logging: ON

PRODUCTION (LOW VOLUME, <1000 CALLS/DAY)

- Provider: Anthropic + API Key (separate billing)
- Model: `claude-sonnet-4-6`
- Timeout: 120s
- Max output: 4096
- PDF pages: 10
- Logging: ON + dashboard query on cost

PRODUCTION (HIGH VOLUME, 10000+ CALLS/DAY)

- Provider: Anthropic + API Key
- Model: `claude-haiku-4-5` as global default (most calls are simple). Override to Sonnet via prompt template for tasks that need it.
- Timeout: 60s (fail fast at scale)
- Max output: 2048
- PDF pages: 10
- Logging: ON, but consider shortening retention to 30 days

4. Pitfalls

1. **Setting a model that doesn't exist in the provider's catalog.** The provider's `models` table is the source of truth. Click "Fetch Models" on the provider doc whenever you change models. The dispatcher will raise if you reference an unknown model.
 2. **Switching the default mid-conversation.** Long-running async jobs use the provider/model snapshot from when they were enqueued, not from current settings. Change defaults during a quiet period.
 3. **Disabling logging while async consumers are listening.** Async callers wait for `frappe.realtime` events tied to the log name. With logging off, they get a random hash and never receive notification of completion.
 4. **Raising timeout without raising worker pool.** A long-running AI call holds a Frappe background worker. If your timeout is 300s and you only have 4 workers, four concurrent slow AI calls = whole queue stalled.
 5. **OAuth tokens without the Claude Code header.** If you configured Anthropic + Auth Token before commit fixing `_get_client()`, requests will 429. The fix adds `anthropic-beta: oauth-2025-04-20` + system-prompt prefix automatically.
-

5. Verifying Your Setup

```
# bench --site your-site.local console
from ai_interface.services.generator import generate
print(generate(prompt="Say hi in 5 words", sync=True, calling_app="smoke_test"))
```

Then in Desk → **AI Call Log** → sort by creation desc. You should see a `Completed` row with non-zero `cost` and `latency_ms`.

If you see `Failed`: open the row, read `error_message`. Common causes:

- 401 → wrong credential. Check provider's `api_key` / `auth_token`.
 - 429 → rate limit. If OAuth: verify the system prompt prefix patch is applied. Otherwise: wait + retry.
 - 404 on model → model ID typo in Default Model field, or provider's `models` table is stale (re-fetch).
-

6. When to Change These Settings vs Per-Call Overrides

Scenario	Where to change
All apps should default to Haiku for cost	AI Settings → Default Model
Only DocPulse should use Opus for OCR	DocPulse code: <code>extract(..., model="claude-opus-4-7")</code>
Email-drafting template needs longer output	AI Prompt Template <code>email_draft</code> → <code>model_override</code> or <code>max_tokens</code> in code
Vision-heavy app needs 300s timeout	Either raise globally, or split that app's calls to async (no perceived latency)
One app processes 100-page contracts	Don't raise Max PDF Pages globally — chunk in that app's code

Rule of thumb: If a setting change benefits every consumer app, put it here. If it benefits one, put it in that app's call site or its templates.